USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Milestone 5 Verification Report

UM-NATOS-013

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-013	1.1 · 2026-08-15	**PASS** — all three exit criteria met on hardware; §8 added, applications can now communicate	Internal

1. Abstract

Milestone 5 runs multiple bytecode applications concurrently, each confined to its own arena, with an application that misbehaves terminated without affecting the others. It adds an application table and lifecycle, a third level of scheduling, and a console shell.

UM-NATOS-007 §7 names this milestone's principal risk plainly: "*an application deliberately written to escape its arena must fail to do so.*" §5.2 is that test, run against a program written for no other purpose.

This completes the roadmap M0–M5.

2. Three levels of scheduling

The system now preempts at three independent levels, and the design point is that **only the newest one knows the others exist**:

Level	Mechanism	Granularity	Introduced
1	Timer interrupt preempts native tasks	Any machine instruction	M2
2	`vm_run()` quantum returns control	Bytecode instruction boundary	M4
3	`app_tick()` round-robins that quantum across applications	One quantum per application	M5

Adding level 3 required **no change to the scheduler and no change to the interpreter**. `app_tick()` is an ordinary function calling `vm_run()` in a loop, hosted by an ordinary native task. Levels 1 and 2 remain unaware that applications exist.

That is the payoff for the boundary drawn in M4: because `vm_t` carries all execution state and `vm_run()` is resumable, multiplexing applications is bookkeeping rather than surgery.

3. Lifecycle

`app_start()` allocates an arena, copies the image in, and initialises a VM. `app_kill()` and the fault path both release the arena.

All three termination routes — halt, fault, kill — funnel through one `retire()` function. Three separate release paths would be three chances to leak an arena, and "terminating an application releases its arena

completely" is an exit criterion measured to the byte in §5.3.

An application publishes a progress word at a known offset in its own arena. The kernel reads it directly. **The kernel can see into an arena; a program cannot see out of one** — that asymmetry is the entire security model, and the progress word makes it visible rather than theoretical.

4. The shell

A native task polling UART0. It holds no privilege the rest of the kernel lacks; it is a front end to `app_start()` and `app_kill()`.

Programs are registered by the caller rather than referenced directly, so the shell has no dependency on which images exist. UART receive was added for it — polled rather than interrupt-driven, because the shell is the only consumer and a console that drops a keystroke under load is a better outcome than a second interrupt source competing with the scheduler tick.

`shell_poll()` never blocks, so a user holding a key cannot starve the system.

5. Results

```
[1] interleave: PASS  a insns=60000 count=19999  |  b insns=60000 square=224970001

  [app 2 'rogue' TERMINATED] out of bounds at offset 256 of 256 B arena,
                             pc=28, after 167 instructions

[2] isolation : PASS  rogue faulted at offset 256 = arena size;
                      neighbours still running and advancing
[3] release   : PASS  heap 158048/158048 B, live=0, check=0

tasks: report=0 a=1 b=2 vm=3 apps=4 shell=5
```

5.1 Criterion 1 — two applications interleave

Both received an identical instruction budget: **60,000 each**. `app_tick()` hands out the same quantum per round, and equal totals confirm neither starved nor over-ran.

Their *progress* differs, which is the more informative result. The published values are exact:

```
A: 19,999 iterations × 3 instructions + 3 preamble = 60,000
B: 14,999 iterations × 4 instructions + 3 preamble = 59,999   (sampled mid-iteration)
   14,999² = 224,970,001 — the published square, exactly
```

B does more work per iteration and therefore advances more slowly in its own terms while consuming the same CPU. Two applications sharing a core should not advance in lockstep, and the differing rates make that observable rather than assumed. That B's published value is exactly $14{,}999^2$ also confirms its arithmetic survived every preemption intact.

5.2 Criterion 2 — the rogue is terminated, alone

`app_rogue` walks a store upward through memory four bytes at a time, starting above its own code so it does not destroy the loop doing the walking.

Every store inside the arena succeeded. The first store past the end faulted, **at offset 256 of a 256-byte arena** — the boundary is exactly where it should be, not approximately. This is a test of *where* the wall stands, not merely that one exists.

The neighbours were unaffected: both still `running`, and both with published values strictly greater than before the rogue was introduced, so they were still making progress rather than merely still alive.

The deeper point is what the rogue could not attempt. A VM address is an offset into its own arena, so there is no value it could load that names another application's memory. Reaching a neighbour is not refused — it is **unrepresentable**. The bounds check exists for the weaker case of a program walking off its own end, which is precisely what was observed.

Relaunching `rogue` from the shell reused slot 2 and terminated it again identically, confirming the lifecycle is repeatable and not a one-shot.

5.3 Criterion 3 — termination releases the arena completely

After killing both applications, free heap returned to **158,048 B — exactly** its pre-test value, with zero live applications and a clean structural check. An exact match rules out a partial release; an approximate one would have indicated a leaked header or a missed coalesce.

5.4 The shell, exercised

Every command driven over the serial line against the running system:

Command	Result
`help`	command list
`progs`	three programs with image and arena sizes
`ps`	table with live counters and the faulted rogue's diagnosis
`run rogue`	`started id=2`, followed by the termination message
`kill 0`	`killed 0, arena released`
`mem`	`free=155952 largest=155952 blocks=4 high_water=5120 check=0`
`bogus`	rejected, not silently ignored

`ps` output during the run:

```
id  name      state     arena   insns      published
0   counter   running   512 B   3812000    1270666
1   squares   running   512 B   3811635    1795557008
2   rogue     faulted   256 B   167        0   [out of bounds @256]
```

5.5 Regression

Six native tasks running concurrently — reporter, two M2 workers, the M4 VM host, the application host, and the shell. Workers' guards intact and `corrupt=0` throughout; stack headroom 483 words. M2, M3 and M4 self-tests all still passing.

6. Metrics

Quantity	Value
Native tasks	6 (of 8 slots)
Applications	4 slots
Scheduling levels	3
Instruction budget per app, criterion 1	60,000 each
Accounting drift	0 for A, 1 for B (mid-iteration sample)
Rogue fault offset	256, of a 256 B arena
Heap after release	158,048 B, exactly baseline
Image size	14,464 B
Heap, current	158,048 B

Heap fell from 167,680 B because `TASK_MAX` rose from 4 to 8, adding 8 KB of statically allocated task stacks, plus the application table and shell buffers. That is a deliberate trade: six concurrent tasks needed the slots.

7. What M5 does not establish

- **No memory protection between native tasks.** Only *applications* are isolated. The six native tasks share one address space with nothing between them, exactly as before. A driver bug can still corrupt the kernel.
- **No per-application CPU accounting or priority.** Every application gets the same quantum, and there is no way to say one matters more.
- **Messaging is one slot deep and unidirectional per send.** §8 adds point-to-point messages, but there is no broadcast, no queue, and no way to wait for a message — a receiver polls.
- **No program loading from storage.** Images are compiled into the kernel. There is no filesystem, so "install an application" has no meaning yet.
- **The shell has no history, editing beyond backspace, or completion.**
- **Fault recovery is termination only.** There is no way for an application to handle its own fault, and no restart policy.
- **Console output interleaves.** The reporter task and the shell share UART0 with no arbitration, so a report can land in the middle of a typed line. It is cosmetic, but it is the first thing that would need a lock or a mutex — and the kernel has neither.

8. Applications can now communicate

Revision 1.1. This report originally recorded that applications had no way to talk to each other at all, and called that "simple but not ultimately useful". Messaging was added afterwards without weakening anything in §5.

```
ipc s/d/r = 278/277/157957      badbuf = 0
```

278 sent and 277 delivered — every accepted message reached its recipient, one in flight when sampled. No application ever offered a buffer outside its own arena.

8.1 Copied, never shared

Applications have no shared memory and were not given any. An arena is the unit of isolation (§5.2), and mapping one into another would dissolve the single property the rest of the system is built on.

Messages are therefore copied **twice**: out of the sender's arena into a kernel mailbox, and later out of that mailbox into the receiver's arena. Two copies of at most 64 bytes is the price; what it buys is that neither application holds a reference to the other's memory, and neither can learn the other's layout.

A sender names a **destination application**, never an address. As with the arena, the viewport and the pointer, the unwanted operation is not refused — it cannot be expressed.

8.2 Refusal over queueing

One mailbox per application, holding one message. A second message to an occupied mailbox is refused and counted.

The 157,957 refusals are not a fault: the sender writes in a tight loop while the receiver drains occasionally. Queueing would need a policy for what to drop when the queue fills, and no consumer exists yet whose requirements would settle that policy. Refusing is the honest placeholder, and the counter makes the back pressure visible rather than hiding it in a buffer.

8.3 Mail that must not outlive its context

Two cases, both of which would otherwise create a channel nobody asked for:

- **Sending to a slot that is not running is refused.** Otherwise the message waits for whoever occupies that slot next — two applications connected without either having agreed to it.
- **Retiring an application clears its mailbox**, so a successor in the same slot cannot read its predecessor's mail.

9. References

- UM-NATOS-007 §7 — M5 deliverable, principal risk, and exit criteria
- UM-NATOS-012 — the VM and its resumable-quantum design, which made level 3 cheap
- UM-NATOS-010 §5.2 — arena bounds and the offset-domain check
- UM-NATOS-009 — the native scheduler underneath all of this
- `kernel/app.c` — table, lifecycle, and the single `retire()` path
- `kernel/shell.c` — console
- `tools/app_rogue.vasm` — the escape attempt of §5.2